5. Python SQLite Game Shop Tutorial - Update

Contents

5. Python SQLite Game Shop Tutorial - Update 1
SQL Tutorials 1
SQLite with Python Tutorials 1
Explanation: Updating Data 1
Do: Update Data with DB Browser for SQLite 2
Do: sqlite_tutorial_5_update.py 3
Assignment Submission 6

- Follow all directions carefully and accurately.
- Think of the directions as minimum requirements.

SQL Tutorials

- https://www.w3schools.com/sql/sql_update.asp

SQLite with Python Tutorials

- SQLite Databases with Python - Full Course – FreeCodeCamp.org
- https://www.sqlitetutorial.net

Explanation: Updating Data

Rows in an SQLite database can be modified using **UPDATE** SQL statement. We can update single columns as well as multiple columns.

This is the general form to update existing table data.

```
UPDATE table_name SET column1 = value1, column2 = value2 WHERE condition;
```

In the above syntax, the **SET** statement is used to set new values to the column. The **WHERE** clause is used to select the rows for which the columns are needed to be updated.

Let's say, for example, that Sammy the shark was moved to tank number 2. We can change Sammy's row in the fish table to reflect this change:

```
new_tank_number = 2
moved_fish_name = "Sammy"
cursor.execute(
    "UPDATE fish SET tank_number = ? WHERE name = ?",
    (new_tank_number, moved_fish_name)
)
```

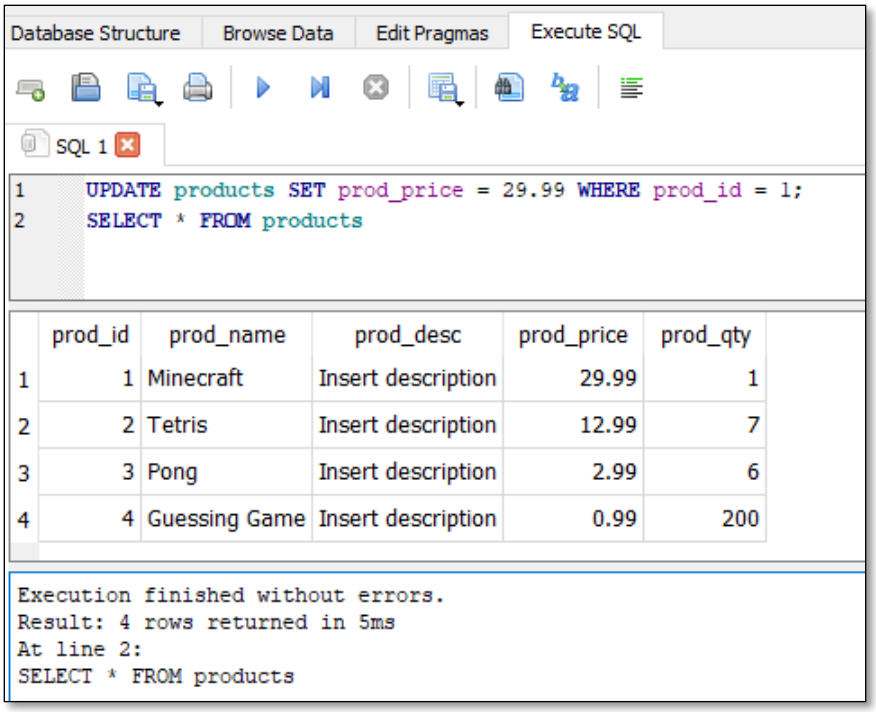
1. We issue an **UPDATE** SQL statement to change the **tank_number** of Sammy to its new value of 2.
2. The **WHERE** clause in the **UPDATE** statement ensures we only change the value of **tank_number** if a row has name = "Sammy".

If we run the following **SELECT** statement, we can confirm our update was made correctly:

```
rows = cursor.execute("SELECT name, species, tank_number FROM fish").fetchall()
print(rows)
```

Do: Update Data with DB Browser for SQLite

1. Copy **game_shop_4.db** to **game_shop_5.db**
2. In **DB Browser for SQLite** → Open Database **game_shop_5.db**
3. Go to the **Execute SQL** tab.
4. Insert and execute the following SQL code.
   We are running 2 statements, we need a ; at the end of the first statement.



Do: sqlite_tutorial_5_update.py

Add the following code to update a record.

```python
# ---------------------- UPDATE RECORD -----------------------------------#
    def update_record(self, prod_price, prod_id):
        """
        Update the price of a specific product in the 'products' table.

        Parameters:
        prod_price (float): The new price to be updated.
        prod_id (int): The ID of the product to be updated.

        """

        # Establish a connection to the database file using with
        with sqlite3.connect(self.database) as connection:

            # Create a cursor object to interact with the database
            cursor = connection.cursor()

            # SQL SELECT statement with prod_price, and prod_id
            # argument substituted for ? ?
            SQL = """
                UPDATE products
                SET prod_price = ?
                WHERE prod_id = ?
                """

            # Execute SQL UPDATE statement with provided price and product ID
            cursor.execute(
                SQL,
                (
                    prod_price,
                    prod_id
                )
            )
```

Modify the main function.

```python
def main():
    # Create a GameShop instance with the specified database file
    game_shop = GameShop("game_shop_4.db")

    # Create the 'products' table in the database
    game_shop.create_table()
    print("Table created")

    # Insert predefined records into the 'products' table
    game_shop.insert_records()
    print("Records inserted")

    # Fetch and display records from the 'products' table
    records = game_shop.fetch_all_records()
    print("\nRecords returned as a list of tuples")

    # Print the entire list of records
    print(records)

    # Print a specific record from the list (index 3)
    print(records[3])

    # Iterate through the list and display records one at a time
    print("Display records one at a time, iterating through list")

    # Loop through returned list of tuples
    for record in records:
        print(record)

    # Fetch all records based on the product price
    print("Fetch filtered records.")
    record = game_shop.fetch_filtered_records(.99)
    print("\nFetch all product that are $.99: ")
    print(record)

    # Update a record's price in the 'products' table based on product ID
    game_shop.update_record(prod_price="29.99", prod_id=1)
    print("\nUpdated sale price for MineCraft")

    # fetch and display all records, showing the update sale price
    records = game_shop.fetch_all_records()
    for record in records:
        print(record)


# If a standalone program, call the main function
# Else, use as a module
if __name__ == "__main__":
    main()
```

Example run:

```
Records returned as a list of tuples
[(1, 'Minecraft', 'Insert description', 15.99, 1, 1), (2, 'Tetris', 'Insert description', 12.99, 3, 7), (
3, 'Pong', 'Insert description', 0.99, 4, 6), (4, 'Guessing Game', 'Insert description', 0.99, 10, 200)]
(4, 'Guessing Game', 'Insert description', 0.99, 10, 200)
Display records one at a time, iterating through list
(1, 'Minecraft', 'Insert description', 15.99, 1, 1)
(2, 'Tetris', 'Insert description', 12.99, 3, 7)
(3, 'Pong', 'Insert description', 0.99, 4, 6)
(4, 'Guessing Game', 'Insert description', 0.99, 10, 200)
Fetch filtered records.

Fetch all product that are $.99:
[(3, 'Pong', 'Insert description', 0.99, 4, 6), (4, 'Guessing Game', 'Insert description', 0.99, 10, 200)
]

Updated sale price for MineCraft
(1, 'Minecraft', 'Insert description', 29.99, 1, 1)
(2, 'Tetris', 'Insert description', 12.99, 3, 7)
(3, 'Pong', 'Insert description', 0.99, 4, 6)
(4, 'Guessing Game', 'Insert description', 0.99, 10, 200)
```

Assignment Submission

1. Attach the program files.
2. Attach screenshots showing the successful operation of the program.
3. Submit in Blackboard.